

1. main.c

Main application — sensor init, buzzer/LED control, pulse monitoring loop

```
#include<lpc21xx.h>
#include <stdio.h>
#include"header.h"

#define grn (1<<16)
#define rd (1<<17)
#define buz (1<<18)
#define max_temp 38
#define min_heart_rate 60
#define max_heart_rate 99
#define min_spo2 90

#define ir_threshold 75000

MAX30102_DATA SAMPLE;
pulse_result PULSE;

int main()
{
    //float cur_temp =0.0,v;
    char s[50],s2[20];
    //unsigned char data[6];
    //int cur_heart_rate=0;
    //int cur_spo2=0;
    //unsigned int id;
    IODIR0=buz||grn||rd;
    uart0_init(9600);
    adc_init();
    lcd_init();
    i2c_init();
    //DHT11_Start();
    pulse_init();

    sprintf(s,"MODE = %0x2x FIFO=%02x SPO2=%02x LED1=%02x LED2=%02x\r\n",Pulse_ReadReg(MAX_MODE_CONFIG),Pulse_ReadReg(MAX_FIFO_CONFIG),
    Pulse_ReadReg(MAX_SPO2_CONFIG),Pulse_ReadReg(MAX_LED1_PA),Pulse_ReadReg(MAX_LED2_PA));
    uart0_tx_str(s);

    algorithm_init();

    //lm35 sensor
    while(1){
        //DHT11_ReemadTperature();
        //    delay_ms(100);
        ///a=adc_read(1);
        ///v=((a*3.3)/1023);
        ///cur_temp= (v-0.5)/0.01;
        ///
        ///sprintf(s," adc = %d volt = %.3f Temp = %.2f\r\n",a,v,cur_temp);
```

```

//uart0_float(cur_temp);
//uart0_tx_str(s);

Pulse_ReadFIFO(&SAMPLE);
//sprintf(s2,"RED = %d  IR = %d\r\n",SAMPLE.red,SAMPLE.ir);
//uart0_tx_str(s2);
//delay_ms(50);
if(SAMPLE.ir > ir_threshold){
if(algorithm_process(SAMPLE.ir,SAMPLE.red,&PULSE)){
sprintf(s,"bpm = %u  spo2 = %.1f\r\n",PULSE.bpm,PULSE.spo2);
uart0_tx_str(s);
delay_ms(1000);
}
}
}
}
}

```

2. Pulse Sensor Register Driver

MAX30102 register read/write wrapper and FIFO sample reader

```

#include "header.h"
void Pulse_WriteReg(unsigned char reg,unsigned char value){
i2c_send(MAX30102_ADDR,reg,value);

}
unsigned char Pulse_ReadReg(unsigned char reg){

return i2c_read(MAX30102_ADDR, reg);

}
void pulse_init(void){

/*reset*/

Pulse_WriteReg(MAX_MODE_CONFIG,0x40);


delay_ms(100);
/*FIFO*/

Pulse_WriteReg(MAX_FIFO_WR_PTR,0x00);

Pulse_WriteReg(MAX_OVF_COUNTER,0x00);

Pulse_WriteReg(MAX_FIFO_RD_PTR,0x00);
/*FIFO CONFIG*/

Pulse_WriteReg(MAX_FIFO_CONFIG,0X0F);
/*SPO2 CONFIG*/

Pulse_WriteReg(MAX_SPO2_CONFIG,0X27);
/*SPO2 MODE*/

```

```

Pulse_WriteReg(MAX_MODE_CONFIG,0X03);
/*LED_CURRENT*/

Pulse_WriteReg(MAX_LED1_PA,0X24);

Pulse_WriteReg(MAX_LED2_PA,0X24);
/*INTERRUPT*/

Pulse_WriteReg(MAX_INTR_ENABLE_1,0X40);

Pulse_WriteReg(MAX_INTR_ENABLE_2,0X00);
}
void Pulse_ReadFIFO(MAX30102_DATA *sample){

unsigned char data[6];
i2c_read_multi(MAX30102_ADDR,MAX_FIFO_DATA,data,6);

//data = i2c_read(MAX30102_ADDR,MAX_FIFO_DATA);

sample->red = ((unsigned long )data[0] << 16) | ((unsigned long)data[1] << 8) | ((unsigned long)data[2]);
sample->ir = ((unsigned long )data[3] << 16) | ((unsigned long )data[4] << 8) | ((unsigned long )data[5]);
/*18 bit adc value*/

sample->red &= 0x03ffff;

sample->ir &= 0x03ffff;

}

```

3. I2C Driver

I2C master init, single-byte write/read, and multi-byte read routines

```

#include "header.h"
void i2c_init(void){
PINSEL0|=0x50;//P0.2->SCL,P0.3->SDA

I2SCLH=I2SCLL=75;//100Kbps freq
/*select master mode*/

I2CONCLR=(1<<2);//AA=0

I2CONSET=(1<<6);//I2EN=1

}
/*i2c byte write frame*/

#define SI ((I2CONSET>>3)&1)

void i2c_write(u8 sa,u8 mr,u8 data){

/*step1:generate start condi*/

```

```

I2CONSET=(1<<5); //STA=1

I2CONCLR=(1<<3); //Clear SI*

while(SI==0); //waiting for start condi
I2CONCLR=(1<<5); //STA=0*
/*step2:send sa+w & check ack*/

I2DAT=sa; //send sa+w

I2CONCLR=(1<<3); //SI=0

while(SI==0);

if(I2STAT==0x20){

    uart0_str("err:sa+w\r\n");

    goto exit;

}
/*step3:send mr_addr & check ack*/

I2DAT=mr; //send mr_addr

I2CONCLR=(1<<3); //SI=0

while(SI==0);

if(I2STAT==0x30){

    uart0_str("err:mr_addr\r\n");

    goto exit;

}

/*step4:send data & check ack*/
I2DAT=data; //send data

I2CONCLR=(1<<3); //SI=0

while(SI==0);

if(I2STAT==0x30){

    uart0_str("err:data\r\n");

    goto exit;

}
/*step5:generate stop condi*/

exit:

```

```

I2CONSET=(1<<4); //STO=1

I2CONCLR=(1<<3); //Clear SI*

}
/*i2c byte read frame*/

u8 i2c_read(u8 sa, u8 mr){

u8 res=0;

/*step1:generate start condi*/

I2CONSET=(1<<5);

I2CONCLR=(1<<3);

while(SI==0);

I2CONCLR=(1<<5);

/*step2:send sa+w & check ack*/

I2DAT=sa^1; //send sa+w

I2CONCLR=(1<<3); //SI=0

while(SI==0);

if(I2STAT==0x20){

    uart0_str("err:sa+w\r\n");

    goto exit;

}

/*step3:send mr_addr & check ack*/

I2DAT=mr; //send mr_addr

I2CONCLR=(1<<3); //SI=0

while(SI==0);

if(I2STAT==0x30){

    uart0_str("err:mr_addr\r\n");

    goto exit;

}

/*step4:generate restart condi*/

```

```

I2CONSET=(1<<5);

I2CONCLR=(1<<3);

while(SI==0);

I2CONCLR=(1<<5);
/*step5:send sa+r & check ack*/

I2DAT=sa;//send SA+R

I2CONCLR=(1<<3);//SI=0

while(SI==0);

if(I2STAT==0x48){

    uart0_str("err:sa+r\r\n");

    goto exit;

}

/*step6:read data from slave & send noack*/

I2CONCLR=(1<<3);//SI=0

while(SI==0);//waiting for data receive

res=I2DAT;

/*step7:generate stop condi*/

exit:

I2CONSET=(1<<4);//STO=1

I2CONCLR=(1<<3);//SI=0

/*step8:return data*/

return res;

}

void i2c_mult_read(u8 sa_add, u8 mr_add, u8 len, u8 *buf)
{
    u8 i;

```

```

/* START */
I2CONSET = (1<<5);
while(SI==0);
I2CONCLR = (1<<5) | (1<<3);

/* Slave Address + Write */
I2DAT = sa_add;
while(SI==0);
I2CONCLR = (1<<3);

if(I2STAT==0x20)
{
    // uart0_str("Slave Address Error\n");
    goto exit;
}

/* Memory Address */
I2DAT = mr_add;
while(SI==0);
I2CONCLR = (1<<3);

if(I2STAT==0x30)
{
    // uart0_str("Memory Address Error\n");
    goto exit;
}

/* Repeated START */
I2CONSET = (1<<5);
while(SI==0);
I2CONCLR = (1<<5) | (1<<3);

/* Slave Address + Read */
I2DAT = sa_add^1;
while(SI==0);
I2CONCLR = (1<<3);

if(I2STAT==0x48)
{
    // uart0_str("Read Address Error\n");
    goto exit;
}

for(i=0;i<len;i++)
{
    if(i < (len-1)) I2CONSET = (1<<2);    // ACK
    else I2CONCLR = (1<<2);    // NACK

    while(SI==0);

    buf[i] = I2DAT;

    I2CONCLR = (1<<3);
}

```

```

        if((i < (len-1)) && (I2STAT!=0x50))
            break;

        if((i==(len-1)) && (I2STAT!=0x58))
            break;
    }

exit:
    I2CONSET = (1<<4);    // STOP
    I2CONCLR = (1<<3);
}

```

4. Heart Rate / SpO2 Algorithm (Moving Average Method)

Moving-average filter based peak detection for BPM and SpO2 estimation

```

#include "header.h"

#include "pulse.h"

#define SAMPLE_RATE 100

#define BUFFER_SIZE 100

#define FILTER_SIZE 8

#define MIN_PEAK_DISTANCE 40

#define MAX_BEATS 5
static unsigned long ir_filter[FILTER_SIZE];

static unsigned char filter_index;

static unsigned long ir_buffer[FILTER_SIZE];

static unsigned long red_buffer[BUFFER_SIZE];

static unsigned short sample_index;
static unsigned long prev_avg = 0;

static unsigned short last_peak = 0;

static unsigned char rising = 0;
static unsigned short beat_interval[MAX_BEATS];

static unsigned char beat_index;
static unsigned long moving_average(unsigned long value)

{
    unsigned long sum=0;

    unsigned char i;

```



```

    ir_filter[filter_index]=value;

    filter_index++;

    if(filter_index>=FILTER_SIZE)

        filter_index=0;

    for(i=0;i<FILTER_SIZE;i++)

        sum+=ir_filter[i];

    return sum/FILTER_SIZE ;

}

void algorithm_init(){
    unsigned short i;
    for(i=0 ; i<        FILTER_SIZE ; i++)

        ir_filter[i] = 0;

    for(i=0 ; i<BUFFER_SIZE ; i++)
    {

        ir_buffer[i] = 0;

        red_buffer[i]=0;

    }
    filter_index =0;

    sample_index=0;

    prev_avg =0;

    beat_index =0;

    last_peak      =0;

    rising =0;

}

unsigned char algorithm_process(unsigned long ir, unsigned long red,pulse_result *result){
    unsigned short i;
        unsigned long avg ;

        unsigned short diff;
        unsigned long ir_max,ir_min;
        unsigned long red_max,red_min;
        float ac_ir;
        float dc_ir;
        float ac_red;
        float dc_red;

```

```

float ratio;
ir_buffer[sample_index] = ir;
red_buffer[sample_index]=red;
avg=moving_average(ir);
sample_index++;

if(sample_index >= BUFFER_SIZE)

sample_index =0;
if (avg>prev_avg)
rising=1;
if ((avg<prev_avg) &&rising)

{

diff=(sample_index>=last_peak)?(sample_index-last_peak):(BUFFER_SIZE-last_peak+sample_index);

if (diff>MIN_PEAK_DISTANCE)

{

beat_interval[beat_index]=diff;

beat_index++;

if (beat_index>=MAX_BEATS)

beat_index=0;

last_peak=sample_index;

rising=0;

diff=0;

for (i=0;i<MAX_BEATS;i++)

diff+=beat_interval[i];

diff/=MAX_BEATS;

result->bpm=(60*SAMPLE_RATE)/diff;

ir_max=0;

ir_min=0xffffffff;

red_max=0;

red_min=0xffffffff;

for (i=0 ; i< BUFFER_SIZE ; i++)
{
if (ir_buffer[i]>ir_max)

```

```

    ir_max=ir_buffer[i];
    if(ir_buffer[i]<ir_min)

    ir_min=ir_buffer[i];
    if(red_buffer[i]>red_max)

    red_max=red_buffer[i];
    if(red_buffer[i]<red_min)

    red_min=red_buffer[i];

}

dc_ir=(ir_max+ir_min)/2.0f;

ac_ir=(ir_max-ir_min);
dc_red=(red_max+red_min)/2.0f;
ac_ir=(red_max-red_min);
ratio=(ac_red/dc_red)/(ac_ir/dc_ir);
result->spo2=110.0f-(25.0f*ratio);
if(result->spo2>100)

result->spo2=100;
if(result->spo2<80)

result->spo2=80;
result->valid=1;

prev_avg=avg;

return 1;

}

    rising=0;

}
    prev_avg=avg;

    result->valid=0;

    return 0;

}

```

5. Heart Rate & SpO2 Algorithm

Autocorrelation-based periodicity detection with linear-trend removal for HR/SpO2

```
#include"header.h"
```

```

void rf_heart_rate_and_oxygen_saturation(u32 *pun_ir_buffer, s32 n_ir_buffer_length, u32 *pun_red_buffer, float *pn_spo2, s8
*pch_spo2_valid, s32 *pn_heart_rate,
                                     s8 *pch_hr_valid, float *ratio, float *correl)
/**
 * \brief      Calculate the heart rate and SpO2 level, Robert Fraczekiewicz version
 * \par        Details
 *             By detecting peaks of PPG cycle and corresponding AC/DC of red/infra-red signal, the xy_ratio for the SPO2 is computed.
 *
 * \param[in]  *pun_ir_buffer          - IR sensor data buffer
 * \param[in]  n_ir_buffer_length      - IR sensor data buffer length
 * \param[in]  *pun_red_buffer         - Red sensor data buffer
 * \param[out] *pn_spo2                - Calculated SpO2 value
 * \param[out] *pch_spo2_valid         - 1 if the calculated SpO2 value is valid
 * \param[out] *pn_heart_rate          - Calculated heart rate value
 * \param[out] *pch_hr_valid           - 1 if the calculated heart rate value is valid
 *
 * \retval     None
 */
{
    s32 k;
    static s32 n_last_peak_interval=LOWEST_PERIOD;
    float f_ir_mean,f_red_mean,f_ir_sumsq,f_red_sumsq;
    float f_y_ac, f_x_ac, xy_ratio;
    float beta_ir, beta_red, x;
    float an_x[BUFFER_SIZE], *ptr_x; //ir
    float an_y[BUFFER_SIZE], *ptr_y; //red

    // calculates DC mean and subtracts DC from ir and red
    f_ir_mean=0.0;
    f_red_mean=0.0;
    for (k=0; k<n_ir_buffer_length; ++k) {
        f_ir_mean += pun_ir_buffer[k];
        f_red_mean += pun_red_buffer[k];
    }
    f_ir_mean=f_ir_mean/n_ir_buffer_length ;
    f_red_mean=f_red_mean/n_ir_buffer_length ;

    // remove DC
    for (k=0,ptr_x=an_x,ptr_y=an_y; k<n_ir_buffer_length; ++k,++ptr_x,++ptr_y) {
        *ptr_x = pun_ir_buffer[k] - f_ir_mean;
        *ptr_y = pun_red_buffer[k] - f_red_mean;
    }

    // RF, remove linear trend (baseline leveling)
    beta_ir = rf_linear_regression_beta(an_x, mean_X, sum_X2);
    beta_red = rf_linear_regression_beta(an_y, mean_X, sum_X2);
    for(k=0,x=-mean_X,ptr_x=an_x,ptr_y=an_y; k<n_ir_buffer_length; ++k,++x,++ptr_x,++ptr_y) {
        *ptr_x -= beta_ir*x;
        *ptr_y -= beta_red*x;
    }

    // For SpO2 calculate RMS of both AC signals. In addition, pulse detector needs raw sum of squares for IR
    f_y_ac=rf_rms(an_y,n_ir_buffer_length,&f_red_sumsq);
    f_x_ac=rf_rms(an_x,n_ir_buffer_length,&f_ir_sumsq);

```

```

// Calculate Pearson correlation between red and IR
*correl=rf_Pcorrelation(an_x, an_y, n_ir_buffer_length)/sqrt(f_red_sumsq*f_ir_sumsq);

// Find signal periodicity
if(*correl>=min_pearson_correlation) {
    // At the beginning of oximetry run the exact range of heart rate is unknown. This may lead to wrong rate if the next call does not find
    the _first_
    // peak of the autocorrelation function. E.g., second peak would yield only 50% of the true rate.
    if(LOWEST_PERIOD==n_last_peak_interval)
        rf_initialize_periodicity_search(an_x, BUFFER_SIZE, &n_last_peak_interval, HIGHEST_PERIOD, min_autocorrelation_ratio, f_ir_sumsq);
    // RF, If correlation os good, then find average periodicity of the IR signal. If aperiodic, return periodicity of 0
    if(n_last_peak_interval!=0)
        rf_signal_periodicity(an_x, BUFFER_SIZE, &n_last_peak_interval, LOWEST_PERIOD, HIGHEST_PERIOD, min_autocorrelation_ratio, f_ir_sumsq,
ratio);
    } else n_last_peak_interval=0;

// Calculate heart rate if periodicity detector was successful. Otherwise, reset peak interval to its initial value and report error.
if(n_last_peak_interval!=0) {
    *pn_heart_rate = (s32)(FS60/n_last_peak_interval);
    *pch_hr_valid = 1;
} else {
    n_last_peak_interval=LOWEST_PERIOD;
    *pn_heart_rate = -999; // unable to calculate because signal looks aperiodic
    *pch_hr_valid = 0;
    *pn_spo2 = -999 ; // do not use SPO2 from this corrupt signal
    *pch_spo2_valid = 0;
    return;
}

// After trend removal, the mean represents DC level
xy_ratio= (f_y_ac*f_ir_mean)/(f_x_ac*f_red_mean); //formula is (f_y_ac*f_x_dc) / (f_x_ac*f_y_dc) ;
if(xy_ratio>0.02 && xy_ratio<1.84) { // Check boundaries of applicability
    *pn_spo2 = (-45.060*xy_ratio + 30.354)*xy_ratio + 94.845;
    *pch_spo2_valid = 1;
} else {
    *pn_spo2 = -999 ; // do not use SPO2 since signal an_ratio is out of range
    *pch_spo2_valid = 0;
}
}

float rf_linear_regression_beta(float *pn_x, float xmean, float sum_x2)
/**
 * \brief      Coefficient beta of linear regression
 * \par        Details
 *              Compute directional coefficient, beta, of a linear regression of pn_x against mean-centered
 *              point index values (0 to BUFFER_SIZE-1). xmean must equal to (BUFFER_SIZE-1)/2! sum_x2 is
 *              the sum of squares of the mean-centered index values.
 *              Robert Fraczekiewicz, 12/22/2017
 * \retval      Beta
 */
{
    float x,beta,*pn_ptr;
    beta=0.0;

```

```

    for(x=-xmean,pn_ptr=pn_x;x<=xmean;++x,++pn_ptr)
        beta+=x*(*pn_ptr);
    return beta/sum_x2;
}

float rf_autocorrelation(float *pn_x, s32 n_size, s32 n_lag)
/**
 * \brief      Autocorrelation function
 * \par        Details
 *              Compute autocorrelation sequence's n_lag's element for a given series pn_x
 *              Robert Fraczekiewicz, 12/21/2017
 * \retval     Autocorrelation sum
 */
{
    int16_t i, n_temp=n_size-n_lag;
    float sum=0.0,*pn_ptr;
    if(n_temp<=0) return sum;
    for (i=0,pn_ptr=pn_x; i<n_temp; ++i,++pn_ptr) {
        sum += (*pn_ptr)*(*pn_ptr+n_lag);
    }
    return sum/n_temp;
}

void rf_initialize_periodicity_search(float *pn_x, s32 n_size, s32 *p_last_periodicity, s32 n_max_distance, float min_aut_ratio, float
aut_lag0)
/**
 * \brief      Search the range of true signal periodicity
 * \par        Details
 *              Determine the range of current heart rate by locating neighborhood of
 *              the _first_peak of the autocorrelation function. If at all lags until
 *              n_max_distance the autocorrelation is less than min_aut_ratio fraction
 *              of the autocorrelation at lag=0, then the input signal is insufficiently
 *              periodic and probably indicates motion artifacts.
 *              Robert Fraczekiewicz, 04/25/2020
 * \retval     Average distance between peaks
 */
{
    s32 n_lag;
    float aut,aut_right;
    // At this point, *p_last_periodicity = LOWEST_PERIOD. Start walking to the right,
    // two steps at a time, until lag ratio fulfills quality criteria or HIGHEST_PERIOD
    // is reached.
    n_lag=*p_last_periodicity;
    aut_right=aut=rf_autocorrelation(pn_x, n_size, n_lag);
    // Check sanity
    if(aut/aut_lag0 >= min_aut_ratio) {
        // Either quality criterion, min_aut_ratio, is too low, or heart rate is too high.
        // Are we on autocorrelation's downward slope? If yes, continue to a local minimum.
        // If not, continue to the next block.
        do {
            aut=aut_right;
            n_lag+=2;
            aut_right=rf_autocorrelation(pn_x, n_size, n_lag);
        } while(aut_right/aut_lag0 >= min_aut_ratio && aut_right<aut && n_lag<=n_max_distance);
    }
}

```

```

    if(n_lag>n_max_distance) {
        // This should never happen, but if does return failure
        *p_last_periodicity=0;
        return;
    }
    aut=aut_right;
}
// Walk to the right.
do {
    aut=aut_right;
    n_lag+=2;
    aut_right=rf_autocorrelation(pn_x, n_size, n_lag);
} while(aut_right/aut_lag0 < min_aut_ratio && n_lag<=n_max_distance);
if(n_lag>n_max_distance) {
    // This should never happen, but if does return failure
    *p_last_periodicity=0;
} else
    *p_last_periodicity=n_lag;
}

void rf_signal_periodicity(float *pn_x, s32 n_size, s32 *p_last_periodicity, s32 n_min_distance, s32 n_max_distance, float min_aut_ratio,
float aut_lag0, float *ratio)
/**
 * \brief      Signal periodicity
 * \par        Details
 *             Finds periodicity of the IR signal which can be used to calculate heart rate.
 *             Makes use of the autocorrelation function. If peak autocorrelation is less
 *             than min_aut_ratio fraction of the autocorrelation at lag=0, then the input
 *             signal is insufficiently periodic and probably indicates motion artifacts.
 *             Robert Fraczekiewicz, 01/07/2018
 * \retval     Average distance between peaks
 */
{
    s32 n_lag;
    float aut,aut_left,aut_right,aut_save;
    bool left_limit_reached=false;
    // Start from the last periodicity computing the corresponding autocorrelation
    n_lag=*p_last_periodicity;
    aut_save=aut=rf_autocorrelation(pn_x, n_size, n_lag);
    // Is autocorrelation one lag to the left greater?
    aut_left=aut;
    do {
        aut=aut_left;
        n_lag--;
        aut_left=rf_autocorrelation(pn_x, n_size, n_lag);
    } while(aut_left>aut && n_lag>=n_min_distance);
    // Restore lag of the highest aut
    if(n_lag<n_min_distance) {
        left_limit_reached=true;
        n_lag=*p_last_periodicity;
        aut=aut_save;
    } else n_lag++;
    if(n_lag==*p_last_periodicity) {
        // Trip to the left made no progress. Walk to the right.

```

```

aut_right=aut;
do {
    aut=aut_right;
    n_lag++;
    aut_right=rf_autocorrelation(pn_x, n_size, n_lag);
} while(aut_right>aut && n_lag<=n_max_distance);
// Restore lag of the highest aut
if(n_lag>n_max_distance) n_lag=0; // Indicates failure
else n_lag--;
if(n_lag==*p_last_periodicity && left_limit_reached) n_lag=0; // Indicates failure
}
*ratio=aut/aut_lag0;
if(*ratio < min_aut_ratio) n_lag=0; // Indicates failure
*p_last_periodicity=n_lag;
}

float rf_rms(float *pn_x, s32 n_size, float *sumsq)
/**
 * \brief      Root-mean-square variation
 * \par        Details
 *             Compute root-mean-square variation for a given series pn_x
 *             Robert Fraczkiewicz, 12/25/2017
 * \retval     RMS value and raw sum of squares
 */
{
    int16_t i;
    float r,*pn_ptr;
    (*sumsq)=0.0;
    for (i=0,pn_ptr=pn_x; i<n_size; ++i,++pn_ptr) {
        r=(*pn_ptr);
        (*sumsq) += r*r;
    }
    (*sumsq)/=n_size; // This corresponds to autocorrelation at lag=0
    return sqrt(*sumsq);
}

float rf_Pcorrelation(float *pn_x, float *pn_y, s32 n_size)
/**
 * \brief      Correlation product
 * \par        Details
 *             Compute scalar product between *pn_x and *pn_y vectors
 *             Robert Fraczkiewicz, 12/25/2017
 * \retval     Correlation product
 */
{
    int16_t i;
    float r,*x_ptr,*y_ptr;
    r=0.0;
    for (i=0,x_ptr=pn_x,y_ptr=pn_y; i<n_size; ++i,++x_ptr,++y_ptr) {
        r+=(*x_ptr)*(*y_ptr);
    }
    r/=n_size;
    return r;
}

```


6. I2C Driver

I2C driver with status-code checked transactions and multi-byte burst read

```
#include "header.h"

#define SI ((I2CONSET>>3)&1)
extern volatile u8 sample_buf[6];

void i2c_init(void)
{
    // *** I2C Bit freq, setting ***
    // Bit freq. = FCLK/(I2SCLL + I2SCLH)
    // (I2SCLL + I2SCLH) = 15000000/100000; setting as 100kBps

    PINSEL0 |= (1<<4) | (1<<6); //SCL = P0.2 & SDA = P0.3
    I2SCLL = (15000000/100000)/2;
    I2SCLH = (15000000/100000)/2;
    I2CONSET = 1<<6 ;
    I2CONCLR = 1<<2;
}

void i2c_burst_read(u32 slave_addr, u32 memory_addr)
{
    //u8 result;
    s8 i,t=0;

    // Generate start condition
    I2CONSET = (1<<5); // STA =1;
    I2CONCLR = (1<<3); // clearing SI flag
    while(SI==0);
    if(I2STAT != 0x08) // checking for start condition
    {
        uart0_tx_string("\r\n ERROR : START condition \r\n");
        goto exit ; //need to change to goto exit or something
    }
    I2CONCLR = (1<<5); // STA =0;

    // Sending SA+W
    I2DAT = slave_addr ; //SA+W
    I2CONCLR = (1<<3); // clearing SI flag
    while(SI==0);
    if(I2STAT != 0x18) // checking for SA+W,ACK
    {
        uart0_tx_string("\r\n ERROR : SA + W failed , I2STAT != 0x18 \r\n");
        goto exit; //need to change to goto exit or something
    }
    if(I2STAT == 0x20) // checking for SA+W,NOT ACK
    {
        uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
        goto exit; //need to change to goto exit or something
    }
}
```

```

// Sending memory addr
I2DAT = memory_addr ;
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x28) // Data/Memory_addr has been transmitted expected status
{
    uart0_tx_string("\r\n ERROR : memory_addr \r\n");
    goto exit ; //need to change to goto exit or something
}
if(I2STAT == 0x30) // Data/Memory_addr has been transmitted; NOT ACK has been received
{
    uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
    goto exit; //need to change to goto exit or something
}

// Sending Restart condition
I2CONSET = (1<<5); // STA =1;
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x10) // Reapeted Start condition failed transmitted
{
    uart0_tx_string("\r\n ERROR : Restart condition \r\n");
    goto exit; //need to change to goto exit or something
}
I2CONCLR = (1<<5); // STA =0;

// Sending SA+R
I2DAT = slave_addr^1 ; //SA+R
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x40) // Expected status
{
    uart0_tx_string("\r\n ERROR : SA + W failed, I2STAT != 0x40 \r\n");
    goto exit ; //need to change to goto exit or something
}
if(I2STAT == 0x48) // checking for SA+W,NOT ACK
{
    uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
    goto exit ; //need to change to goto exit or something
}
}
for(i=0;i<5;i++)
{

// Geting data & sending ACK
I2CONSET=(1<<2); // seting AA=1
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT == 0x58) // Data byte has been received; NOT ACK has been returned
{
    uart0_tx_string("\r\n ERROR : Data byte has been received; NOT ACK has been returned \r\n");

```

```

        goto exit ; //need to change to goto exit or something
    }
    if(I2STAT != 0x50) // Data byte has been received; ACK has been returned
    {
        uart0_tx_string("\r\n ERROR : ACK \r\n");
        goto exit ; //need to change to goto exit or something
    }

//    uart0_tx_string("\r\n llll Byte = ");
//    uart0_hex( I2DAT);
    sample_buf[t++] = I2DAT;

}

//getting last data byte
I2CONCLR = 1<<2;
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x58) // Data byte has been received; NOT ACK has been returned
{
    uart0_tx_string("\r\n ERROR : DATA reception \r\n");
    goto exit ; //need to change to goto exit or something
}
//    uart0_tx_string("\r\n last Byte = ");
//    uart0_hex( I2DAT);
    sample_buf[t++] = I2DAT;
    sample_buf[t] = 0;
    exit:
    I2CONSET = (1<<4);
    I2CONCLR = (1<<3);
    while((I2CONSET>>4&1) == 1);
        //uart0_tx_string("\r\n $$$$$$ READ COMPLETE $$$$$$...\r\n");

}

void i2c_write(u32 slave_addr, u32 memory_addr , u8 data)
{
    // Generate start condition

    I2CONSET = (1<<5); // STA =1;
    I2CONCLR = (1<<3); // clearing SI flag
    while(SI==0);
    if(I2STAT != 0x08) // checking for start condition
    {
        uart0_tx_string("\r\n ERROR : START condition \r\n");
        goto exit ; //need to change to goto exit or something
    }
    I2CONCLR = (1<<5); // STA =0;

    // sending SA+W
    I2DAT = slave_addr ; //SA+W

```

```

I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);

//          uart0_tx_string("\r\n I2STAT = ");
//          uart0_intiger(I2STAT);
//          uart0_tx_string("\r\n");

if(I2STAT != 0x18) // checking for SA+W,ACK
{
    uart0_tx_string("\r\n ERROR : SA + W failed , I2STAT != 0x18 \r\n");
    goto exit ; //need to change to goto exit or something
}

if(I2STAT == 0x20) // checking for SA+W,NOT ACK
{
    uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
    goto exit ; //need to change to goto exit or something
}

// Sending memory addr

I2DAT = memory_addr ;
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x28)    // Data/Memory_addr has been transmitted expected status
{
    uart0_tx_string("\r\n ERROR : memory_addr \r\n");
    goto exit ; //need to change to goto exit or something
}
if(I2STAT == 0x30)    // Data/Memory_addr has been transmitted; NOT ACK has been received
{
    uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
    goto exit ; //need to change to goto exit or something
}

//Send data
I2DAT = data ;
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x28)    // Expected status
{
    uart0_tx_string("\r\n ERROR : Data , I2STAT != 0x28 \r\n");
    goto exit ; //need to change to goto exit or something
}
if(I2STAT == 0x30)    // Data/Memory_addr has been transmitted; NOT ACK has been received
{
    uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
    goto exit ; //need to change to goto exit or something
}

    exit:
I2CONSET = (1<<4);
I2CONCLR = (1<<3);
while((I2CONSET>>4&1) == 1);
    //uart0_tx_string("\r\n !!!!! Write COMPLETE !!!!!...\r\n");

```

```

}

u8 i2c_read(u32 slave_addr, u32 memory_addr)
{
    u8 result;

    // Generate start condition
    I2CONSET = (1<<5); // STA =1;
    I2CONCLR = (1<<3); // clearing SI flag
    while(SI==0);
    I2CONCLR = (1<<5); // STA =0;

    // Sending SA+W
    I2DAT = slave_addr ; //SA+W
    I2CONCLR = (1<<3); // clearing SI flag
    while(SI==0);
    if(I2STAT != 0x18) // checking for SA+W,ACK
    {
        uart0_tx_string("\r\n ERROR : SA + W failed , I2STAT != 0x18 \r\n");
        goto exit; //need to change to goto exit or something
    }
    if(I2STAT == 0x20) // checking for SA+W,NOT ACK
    {
        uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
        goto exit; //need to change to goto exit or something
    }

    // Sending memory addr
    I2DAT = memory_addr ;
    I2CONCLR = (1<<3); // clearing SI flag
    while(SI==0);
    if(I2STAT != 0x28) // Data/Memory_addr has been transmitted expected status
    {
        uart0_tx_string("\r\n ERROR : memory_addr \r\n");
        goto exit ; //need to change to goto exit or something
    }
    if(I2STAT == 0x30) // Data/Memory_addr has been transmitted; NOT ACK has been received
    {
        uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
        goto exit; //need to change to goto exit or something
    }

    // Sending Restart condition
    I2CONSET = (1<<5); // STA =1;
    I2CONCLR = (1<<3); // clearing SI flag
    while(SI==0);
    if(I2STAT != 0x10) // Reapeted Start condition failed transmitted
    {
        uart0_tx_string("\r\n ERROR : Restart condition \r\n");
        goto exit; //need to change to goto exit or something
    }
}

```

```

I2CONCLR = (1<<5); // STA =0;

// Sending SA+R
I2DAT = slave_addr^1 ; //SA+R
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x40) // Expected status
{
    uart0_tx_string("\r\n ERROR : SA + W failed, I2STAT != 0x40 \r\n");
    goto exit ; //need to change to goto exit or something
}
if(I2STAT == 0x48) // checking for SA+W,NOT ACK
{
    uart0_tx_string("\r\n ERROR : SA + W failed , NOT ACK \r\n");
    goto exit ; //need to change to goto exit or something
}

// Geting data
I2CONCLR = 1<<2;
I2CONCLR = (1<<3); // clearing SI flag
while(SI==0);
if(I2STAT != 0x58) // checking for 0x58 status
{
    uart0_tx_string("\r\n ERROR : DATA reception \r\n");
    goto exit ; //need to change to goto exit or something
}
result = I2DAT;
exit:
//      I2CONSET = (1<<4);
//      I2CONCLR = (1<<3);
//      delay_ms(100);
I2CONSET = (1<<4);
I2CONCLR = (1<<3);
while((I2CONSET>>4&1) == 1);
    //uart0_tx_string("\r\n $$$$ READ COMPLETE $$$$...\r\n");
return result;

}

```

7. MAX30102 Sensor Driver

Sensor initialization, temperature reading, and warning handler

```

#include"header.h"

// MAX30102
volatile u8 sample_buf[6];

extern u32 ir_buffer[MY_BUFFER_SIZE], red_buffer[MY_BUFFER_SIZE];
static u32 index=0;

```

```

//////// WARNING //////////

void warning (void)
{
    IOCLR0 = WARNING_LED;
    IOCLR0 = BUZZER;
    lcd_cmd(0x01);
    lcd_cmd(0x80);
    lcd_str("WARNING");
    //can use an switch interrupt here to reset from this while(sw==0); inside isr flag=1;
}

void MAX_init(void)
{
    uart0_tx_string("\r\n IN MAX_init...\r\n");

    // mode configartion 0x09
    i2c_write(0xAE,0x09,0x40);
    do
    {
        uart0_tx_string("\r\n Checking power ready \r\n");
        uart0_tx_string("\r\n Power ready = ");
        uart0_hex(i2c_read(0xae,0x00));
    }while(i2c_read(0xae,0x09) & 0x40);

    // FIFO configuration(0x08)
    //i2c_write(0xAE,0x08,0xB0);    //Sample averaging = 32 samples, FIFO rollover enabled
    i2c_write(0xAE,0x08,0x00);    //Sample averaging = 0 samples, FIFO rollover disabled

    // spo2 configuration 0x0A
    i2c_write(0xAE,0x0A,0x27);

    // LED pulse amplitude ( 0x0C - 0x0D)
    i2c_write(0xAE,0x0C,0x3F);
    i2c_write(0xAE,0x0D,0x24);

    i2c_write(0xAE,0x09,0x03);

    // clearing the FIFO_WR_PTR, OVF_COUNTER, and FIFO_RD_PTR
    i2c_write(0xAE,0x04,0x00);
    //uart0_tx_string("\r\n read_FIFO_WR_PTR = ");
    //uart0_hex(read_FIFO_WR_PTR());
    //
    //uart0_tx_string("\r\n read_OVR_COUNTER = ");
    i2c_write(0xAE,0x05,0x00);
    //uart0_hex(read_OVR_COUNTER());

    i2c_write(0xAE,0x06,0x00);
    //uart0_tx_string("\r\n read_FIFO_RD_PTR = ");
    //uart0_hex(read_FIFO_RD_PTR());

```

```

//
//uart0_tx_string("\r\n Power ready = ");
//uart0_hex(i2c_read(0xae,0x00));

// Interrupt *****

i2c_write(0xAE,0x02,0xE0);
i2c_write(0xAE,0x03,0x02);

uart0_tx_string("\r\nMAX_init completed...\r\n");
}

u8 sp02_sample_temp(void)
{
    //u8 wr_ptr,rd_ptr , no_of_available_samples,full=0;
    u32 red_raw, IR_raw ;
    u8 i=0;

//uart0_tx_string("\r\n read_FIFO_WR_PTR = ");
//uart0_intiger(read_FIFO_WR_PTR());
//
//uart0_tx_string("\r\n read_OVR_COUNTER = ");
//uart0_intiger(read_OVR_COUNTER());
//
//uart0_tx_string("\r\n read_FIFO_RD_PTR = ");
//uart0_intiger(read_FIFO_RD_PTR());

    while(i<32)
    {
        i2c_burst_read(0xAE,0x07);
        red_raw = (sample_buf[0] & 0x03)<<16 | (sample_buf[1]<<8) | sample_buf[2] ;
        IR_raw = (sample_buf[3] & 0x03)<<16 | (sample_buf[4]<<8) | sample_buf[5] ;
        uart0_tx_string("\r\n red_raw : ");
        uart0_intiger(red_raw);
        uart0_tx_string(" | IR_raw : ");
        uart0_intiger(IR_raw);

        ir_buffer[index]=IR_raw;
        red_buffer[index]=red_raw;

        index++;
        if(index==(MY_BUFFER_SIZE))
        {
            uart0_tx_string("\r\n buffer =400 \r\n ");
            index=0;
            return 1;
        }
        i++;
    }
//uart0_tx_string("\r\n index : ");

```



```

//uart0_intiger(index);
    return 0;
}

u8 sp02_sample(void)
{
    u8 wr_ptr,rd_ptr , no_of_available_samples,full=0;
    u32 red_raw, IR_raw ;

    uart0_tx_string("\r\n ***** in sample... *****\r\n");

    wr_ptr = read_FIFO_WR_PTR();
    rd_ptr = read_FIFO_RD_PTR();

    if( wr_ptr > rd_ptr)
    {
        uart0_tx_string("\r\n Inside if \r\n ");
        no_of_available_samples = wr_ptr - rd_ptr;
    }
    else if ( wr_ptr < rd_ptr )
    {
        uart0_tx_string("\r\n Inside else if \r\n ");
        no_of_available_samples = 32 + wr_ptr - rd_ptr;
    }
    else
    {
        uart0_tx_string("\r\nINT1 = ");
        uart0_hex(i2c_read(0xAE, 0x00));

        uart0_tx_string("\r\nWR = ");
        uart0_intiger(read_FIFO_WR_PTR());

        uart0_tx_string("\r\nRD = ");
        uart0_intiger(read_FIFO_RD_PTR());
        uart0_tx_string("\r\n Inside else \r\n ");
        return 0;
    }

    uart0_tx_string("\r\n wr_ptr = ");
    uart0_intiger(wr_ptr);
    uart0_tx_string("\r\n rd_ptr = ");
    uart0_intiger(rd_ptr);
    uart0_tx_string("\r\n no. of samples = ");
    uart0_intiger(no_of_available_samples);

    while(no_of_available_samples)
    {

        i2c_burst_read(0xAE,0x07);
        red_raw = (sample_buf[0] & 0x03)<<16 | (sample_buf[1]<<8) | sample_buf[2] ;
        IR_raw = (sample_buf[3] & 0x03)<<16 | (sample_buf[4]<<8) | sample_buf[5] ;
        //  uart0_tx_string("\r\n red_raw : ");

```

```

//  uart0_intiger(red_raw);
//  uart0_tx_string(" | IR_raw : ");
//  uart0_intiger(IR_raw);

    ir_buffer[index]=IR_raw;
    red_buffer[index]=red_raw;

index++;
if(index>=(MY_BUFFER_SIZE))
{
    uart0_tx_string("\r\n buffer =600 \r\n ");
    index=0;
    full =1;
}

    no_of_available_samples -- ;
}
uart0_tx_string("\r\n index : ");
uart0_intiger(index);
    return full;
}

u8 temp(void)
{
    u8 temp_fraction;
    i2c_write(0xAE,0x21,0x01);
    while( i2c_read(0xAE,0x21) &1);
    temp_fraction = i2c_read(0xAE,0x20);
    return ( i2c_read(0xAE,0x1F) + ( temp_fraction & 0x0F) );
}

u8 read_FIFO_WR_PTR(void)
{
    u8 res;
    res = i2c_read(0xAE,0x04);
    return  res;//& 0x0f;
}

u8 read_OVR_COUNTER(void)
{
    u8 res;
    res = i2c_read(0xAE,0x05);
    return  res ;//& 0x0f;
}

u8 read_FIFO_RD_PTR(void)
{
    u8 res;
    res = i2c_read(0xAE,0x06);
    return  res ;//& 0x0f;
}

```

```

}

u32 average(u32 * b)
{
    u32 av=0,i;

    for( i=0;i<MY_BUFFER_SIZE;i++)
    {
        av=av+b[i];
    }
    return av/MY_BUFFER_SIZE;
}

```

8. Main Application — ThingSpeak Upload

Main loop with HR/SpO2 display, and ThingSpeak data upload

```

#include"header.h"

volatile u8 timer_flag;
#define LED 1<<3

//u32 ir_buffer[MY_BUFFER_SIZE], red_buffer[MY_BUFFER_SIZE];

u32 ir_buffer[100] = {
90000, 90040, 90080, 90120, 90160, 90200, 90240, 90280, 90320, 90360,
90400, 90440, 90480, 90520, 90560, 90600, 90640, 90680, 90720, 90760,
90800, 90840, 90880, 90920, 90960, 91000, 90960, 90920, 90880, 90840,
90800, 90760, 90720, 90680, 90640, 90600, 90560, 90520, 90480, 90440,
90400, 90360, 90320, 90280, 90240, 90200, 90160, 90120, 90080, 90040,
90000, 90040, 90080, 90120, 90160, 90200, 90240, 90280, 90320, 90360,
90400, 90440, 90480, 90520, 90560, 90600, 90640, 90680, 90720, 90760,
90800, 90840, 90880, 90920, 90960, 91000, 90960, 90920, 90880, 90840,
90800, 90760, 90720, 90680, 90640, 90600, 90560, 90520, 90480, 90440,
90400, 90360, 90320, 90280, 90240, 90200, 90160, 90120, 90080, 90040
};

u32 red_buffer[100] = {
70000, 70020, 70040, 70060, 70080, 70100, 70120, 70140, 70160, 70180,
70200, 70220, 70240, 70260, 70280, 70300, 70320, 70340, 70360, 70380,
70400, 70420, 70440, 70460, 70480, 70500, 70480, 70460, 70440, 70420,
70400, 70380, 70360, 70340, 70320, 70300, 70280, 70260, 70240, 70220,
70200, 70180, 70160, 70140, 70120, 70100, 70080, 70060, 70040, 70020,
70000, 70020, 70040, 70060, 70080, 70100, 70120, 70140, 70160, 70180,
70200, 70220, 70240, 70260, 70280, 70300, 70320, 70340, 70360, 70380,
70400, 70420, 70440, 70460, 70480, 70500, 70480, 70460, 70440, 70420,
70400, 70380, 70360, 70340, 70320, 70300, 70280, 70260, 70240, 70220,
70200, 70180, 70160, 70140, 70120, 70100, 70080, 70060, 70040, 70020
};

int main()
{
//u32 i;

```

```

float spo2;
float ratio;
float correlation;
s32 heart_rate , temp_sensor , count;
s8 hr_valid, spo2_valid ;

IODIR0 = WARNING_LED;


uart0_init(9600);
i2c_init();
MAX_init();
adc_init();

uart0_tx_string("\r\n Started \r\n " );

    lcd_cmd(0x01);
    lcd_cmd(0x80);
    lcd_str("Starting....");

////**** Set wifi return status , if not connected try again , set try again limit ( only connected then move forward) .. etc
    lcd_cmd(0x01);
    lcd_cmd(0x80);
    lcd_str("Searching WIFI");
    count = 0;
while((wifi_start())==0)
{
if( ++count == 10)
{
    lcd_cmd(0x01);
    lcd_cmd(0x80);
    lcd_str("ERROR : WIFI");
    lcd_cmd(0xC0);
    lcd_str("Not Connected");
    break;
}
}
if(count < 10)
{
    lcd_cmd(0x01);
    lcd_cmd(0x80);
    lcd_str("WIFI CONECTED");
    lcd_cmd(0xC0);
    lcd_str("SUCCESSFUL");
}
////////////////////////////////////

while(1)
{

```

```

if(sp02_sample_temp())
{
    if (ir_buffer[0] < 50000)
    {
        uart0_tx_string("\r\n Sensor not in use. Please place your finger.");
        lcd_cmd(0xC0);
        lcd_str("NO SENSOR");
    }
    else
    {
        rf_heart_rate_and_oxygen_saturation( ir_buffer, BUFFER_SIZE, red_buffer, &spo2, &spo2_valid, &heart_rate, &hr_valid, &ratio,
        &correlation );

        /// Heart rate

        if(hr_valid)
        {
            uart0_tx_string("\r\n HR = " );
            uart0_intiger(heart_rate); //printf("HR = %d", heart_rate);
            lcd_cmd(0x80);
            lcd_str("HR : ");
            lcd_integer(heart_rate);
        }
        else
        {
            uart0_tx_string("\r\n Invalid HR \r\n " );
            warning();
            lcd_cmd(0xC0);
            lcd_str("INVALID HR");
        }

        /// spo2

        if(spo2_valid)
        {
            uart0_tx_string("\r\n SPO2 = " );
            uart0_float(spo2); //printf("SpO2 = %.1f", spo2);
            lcd_cmd(0xC0);
            lcd_str("SPO2 : ");
            lcd_float(spo2);
        }
        else
        {
            uart0_tx_string("\r\n Invalid SPO2 \r\n " );
            warning();
            lcd_cmd(0xC0);
            lcd_str("INVALID SPO2");
        }

        // sending data to THINKSPEAK
        count=0;
    }
}

```

```

        if(timer_flag)
        {
            timer_flag=0;
            if((send_to_Thinkspeak(heart_rate, spo2 , temp_sensor)==0))
            {
                uart0_tx_string("\r\n !!!!! ERROR : Unable to send data to thinkspeak !!!!!\r\n "
);
            }
        }
    }

    delay_ms(1000);
}
}

```

9. LCD Driver

4-bit LCD data/command routines, string, integer, and float display functions

```

#include"header.h"

void lcd_data( unsigned char data)
{ //higher nibble
unsigned int t;
IOCLR1=0xfe<<16;
t=(data & 0xf0)<<16;
IOSET1=t;
IOSET1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;

//Lower nibble
IOCLR1=0xfe<<16;
t=(data & 0x0f)<<20;
IOSET1=t;
IOSET1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;
}

void lcd_cmd( unsigned char cmd)
{
unsigned int t;

IOCLR1=0xfe<<16;
t=(cmd & 0xf0)<<16;
IOSET1=t;

```

```

IOCLR1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;

IOCLR1=0xfe<<16;
t=(cmd & 0x0f)<<20;
IOSET1=t;
IOCLR1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;
}

void lcd_init(void)
{
    IODIR1=0xfe<<16;
    PINSEL2|=0x0;
    lcd_cmd(0x02);
    lcd_cmd(0x28);
    lcd_cmd(0x0e);
    lcd_cmd(0x01);
    //lcd_cmd(0x0f);
}

void lcd_str(s8 *p)
{
    while(*p)
    {
        lcd_data(*p);
        p++;
    }
}

int str_len(char *p)
{
    int i;
    for(i=0;*p;p++,i++);
    return i;
}

void float1(float f)
{
    int num,dec,i,j=0,a[20],t,s;
    num=(int)f;
    dec=(f-num)*1000000;
    for(i=dec;i=i/10,j++);
    for(i=0;j;j--,i++,dec=dec/10)
    {
        a[i]=dec%10 + 48;
    }
    a[i]='.';

```

```

t++;
for(i=num; i; i=i/10, j++) {}
for(i=t, s=j; j; j--, i++, num=num/10)
{
a[i]=num%10 + 48;
}

for(i=6+s; i>=0; i--)
    lcd_data(a[i]);
}

void lcd_integer(int num)
{
    int i, a[10];
    if(num==0)
        lcd_data('0');
    if(num<0)
    {
        num=-num;
        lcd_data('-');
    }
    for(i=0; num; num=num/10, i++)
    {
        a[i]=num%10 +48;
    }
    for(i=i-1; i>=0; i--)
        lcd_data(a[i]);
}

void lcd_float(float num)
{
    u32 intnum, decinum;
    s32 i=0;
    u8 a[10];
    if(num<0)
    {
        uart0_tx_string("\r\n LCD : Num is -ve\r\n");
        //str_len("Num is -ve");
        return;
    }
    if(num==0)
    {
        lcd_data('0');
        return;
    }
    intnum=num;
    decinum=(num-intnum)*10;

    if(intnum==0)
    {
        lcd_data('0');
    }
    else {

```



```

        while(intnum)
        {
            a[i]=intnum%10;
            intnum/=10;
            i++;
        }
        while(i>0)
        {
            i--;
            lcd_data(a[i]+48);
        }
        lcd_data('.');

        if(decinum==0)
        {
            lcd_data('0');
            return;
        }
        else {

            while(decinum)
            {
                a[i]=decinum%10;
                decinum/=10;
                i++;
            }

            while(i>0)
            {
                i--;
                lcd_data(a[i]+48);
            }
        }
    }
}

```

10. UART0 Driver

UART0 init, RX/TX, string/float/integer/hex transmit and receive helpers

```

#include"header.h"

void uart0_init(unsigned int baud)
{
    unsigned int pclk,d;
    if (VPBDIV==0)
        pclk=15000000;
    else if (VPBDIV==1)
        pclk=60000000;
    else if (VPBDIV==2)
        pclk=30000000;

    PINSEL0 |=0x05;
    d=pclk/(16*baud);
}

```

```

U0LCR=0x83;
U0DLL=d&0xff;
U0DLM=(d>>8)&0xff;
U0LCR=0x03;
}

void uart0_rx_buffer_ESPwifi(u8 *buf)
{
    int i=0;
    while(1)
    {
        buf[i]=uart0_rx();

        if(i>3)
        {
            if(buf[i]=='\n' && buf[i-1]=='\r')
            {
                break;
            }
        }

        i++;
    }

    buf[i]=0;
}

void uart0_tx(unsigned char data)
{
    U0THR=data;
    while(((U0LSR>>5)&1)==0);
}

unsigned char uart0_rx(void)
{
    while(((U0LSR>>0)&1)==0);
    return U0RBR;
}

void uart0_tx_string(char *p)
{
    while(*p)
    {
        uart0_tx(*p);
        //while((U0LSR>>5&1)==0);
        p++;
    }
}

void uart0_rx_string(unsigned char * s)
{
    int i=0;
    unsigned char t ;
    while((t=uart0_rx())!='\r')

```

```

{
    uart0_tx(t);
    s[i]=t;
    i++;
}
s[i]=0;
}
void uart0_float(float num)
{
    u32 intnum,decinum;
    s32 i=0;
    u8 a[7];
    if(num<0)
    {
        uart0_tx_string("\r\n Num is -ve\r\n");
        return;
    }
    if(num==0)
    {
        uart0_tx('0');
        return;
    }
    intnum=num;
    decinum=(num-intnum)*10;

    if(intnum==0)
    {
        uart0_tx('0');
    }
    else {
        while(intnum)
        {
            a[i]=intnum%10;
            intnum/=10;
            i++;
        }
        while(i>0)
        {
            i--;
            uart0_tx(a[i]+48);
        }
    }
    uart0_tx('.');

    if(decinum==0)
    {
        uart0_tx('0');
        return;
    }
    else {

        while(decinum)
        {
            a[i]=decinum%10;

```

```

        decinum/=10;
        i++;
    }

    while(i>0)
    {
        i--;
        uart0_tx(a[i]+48);
    }
}

void uart0_intiger(s32 num)
{
    u32 a[10];
    s32 i=0;
    if(num < 0 || num == 0)
    {
        if( num == 0)
        {
            uart0_tx('0');
            return;
        }

        uart0_tx_string("\r\n Negative number \r\n");
        return;
    }
    while(num)
    {
        a[i]=num%10;
        num/=10;
        i++;
    }
    while(i)
    {
        --i;
        uart0_tx(a[i]+48);
    }
}

u32 uart0_rx_string_decimal(void)
{
    s32 i=0;
    u32 d=0;
    u8 t,s[32];
    while((t=uart0_rx())!='\r')
    {
        uart0_tx(t);
        s[i]=t;
        i++;
    }
    s[i]=0;
    i=0;

```

```

while(s[i])
{
    d=(d*10)+(s[i]-48);
    i++;
}
return d;
}

void uart0_hex(u8 num) //for 1 byte
{
    u8 t;
    t = (num>>4&0x0f);

    if(t<10)
    {
        uart0_tx(t+48);
    }
    else
    {
        uart0_tx(t-10+65);
    }
    t = (num&0x0f);

    if(t<10)
    {
        uart0_tx(t+48);
    }
    else
    {
        uart0_tx(t-10+65);
    }
}

```

11. Saline Driver

```

#include<lpc21xx.h>
#include"header.h"
#define TRIG_PIN (1<<20)
#define ECHO_PIN (1<<23)
int trig;
void ultrasonic_init(void)
{
    IODIR0|=TRIG_PIN;
    IODIR0&=~(ECHO_PIN);
    T0TCR=2;
    T0PR=59;
    T0TCR=0;
}

```

```

unsigned int distance(void)
{
    unsigned int time=0;
    unsigned int dis=0;
    unsigned int val=0;
    IOSET0=TRIG_PIN;
    delay_us(10);
    IOCLR0=TRIG_PIN;
    uart0_str("abcd \r\n ");
    while ((IOPIN0 & (ECHO_PIN))==0);
    uart0_str("fghi\r\n ");
    T0TCR=0X02;
    T0TC=0;
    T0TCR=0X01;
    uart0_str("pqrs \r\n ");
    val=(IOPIN0&(ECHO_PIN));
    uart0_str("\r\nval:");
    uart0_int(val);
    while (!(IOPIN0 & (ECHO_PIN)));
    uart0_str("mnop \r\n ");
    T0TCR=0X00;
    time=T0TC;
    dis=time/58;
    //dis=((c*2.0)/58)/30.48;
    return dis;
}

```

12. External Interrupt

```

#include"header.h"
//extern u32 flag;
void eint0_handler()__irq
{
    //emergence();
    EXTINT=1;
    //flag=1;
    VICVectAddr=0;
}

```

```

}

void config_eint0()
{
    PINSEL1|=0x1;
    EXTMODE=1;
    EXTPOLAR=0;
    VICIntSelect=0;
    VICVectCntl0=14|(1<<5);
    VICVectAddr0=(u32)eint0_handler;
    VICIntEnable=1<<14;
}

```

13. DHT11 driver

```

#include "header.h"

unsigned char RH_Int;
unsigned char RH_Dec;
unsigned char Temp_Int;
unsigned char Temp_Dec;
unsigned char CheckSum;
unsigned int timeout;
#define DHT_PIN 11
void DHT11_init(void)
{
    IODIR0 |= (1<<DHT_PIN);
    IOSET0 |= (1<<DHT_PIN);
}

unsigned char DHT11_Start(void)
{
    /* Configure as Output */
    IODIR0 |= (1<<DHT_PIN);
    /* Pull Low for 18ms */
    IOCLR0 = (1<<DHT_PIN);
    delay_ms(18);
    /* Pull High for 30us */
    IOSET0 = (1<<DHT_PIN);
    delay_us(30);
}

```

```

    /* Configure as Input */
    IODIR0 &= ~(1<<DHT_PIN);
    timeout = 10000;
    while((IOPIN0 & (1<<DHT_PIN)) && timeout--);
    if(timeout == 0) return 0;
    timeout = 10000;
    while(!(IOPIN0 & (1<<DHT_PIN))) && timeout--);
    if(timeout == 0) return 0;
    timeout = 10000;
    while((IOPIN0 & (1<<DHT_PIN)) && timeout--);
    if(timeout == 0) return 0;
    return 1;
}

unsigned char DHT11_ReadByte(void)
{
    unsigned char i;
    unsigned char data = 0;
    for(i=0;i<8;i++)
    {
        timeout = 1000;
        while(!(IOPIN0 & (1<<DHT_PIN))) && timeout--);
        if(timeout == 0)
            return 0xFF;
        delay_us(40);
        if(IOPIN0 & (1<<DHT_PIN))
            data = (data<<1) | 1;
        else
            data = (data<<1);
        timeout = 1000;
        while((IOPIN0 & (1<<DHT_PIN)) && timeout--);
        if(timeout == 0)
            return 0xFF;
    }
    return data;
}

```


13. Delay

```
#include"header.h"
void delay_ms(u32 ms)
{
    T0PR=15000-1;
    T0PC=0;
    T0TC=0;
    T0TCR=1;
    while (T0TC<ms);
    T0TCR=0;

}
void delay_sec(u32 sec)
{
    T0PR=15000000-1;
    T0PC=0;
    T0TC=0;
    T0TCR=1;
    while (T0TC<sec);
    T0TCR=0;

}
void delay_us(u32 us)
{
    T0PR=15-1;
    T0PC=0;
    T0TC=0;
    T0TCR=1;
    while (T0TC<us);
    T0TCR=0;

}
```